Mobile Demo Script Explanation -- Gnirts (Reversing)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: this challenge's flag is checked in five independent pieces, each against its own target MD5 hash and its own allowed character set, recovered by brute-forcing each piece separately.

Background -- what the terms below mean

What does "Reversing" mean, as a category? A **Reversing** challenge gives you someone else's already-built app (an APK) and asks you to figure out, by reading its code, some secret value it's hiding -- a PIN, a password, a flag string.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**).

Why is there no exploit app to build here? Reversing challenges usually aren't solved by writing an Android app at all -- they're solved by reading the decompiled logic and then writing a small script (usually Python) that re-implements or defeats that same logic on a regular computer, offline.

What does "constraint-based brute force" mean? Instead of guessing the entire flag at once (which would take far too long), each of the five segments is brute-forced **on its own**, each with a smaller, known character set -- which is fast enough to actually finish, then the five results are joined back together.

How the APK was obtained

Gnirts shares its target APK and its whole analysis folder with **BabyRev** and **Pincode** -- all three were extracted from the same decompile pass. There is no separate Gnirts APK folder of its own.

How the solution was built

No Android app was built. The target was decompiled with jadx to recover each segment's target hash and character set, then a Python script brute-forced each of the five segments independently using multiprocessing, and the results were joined into the final flag.

Files to open

The brute-force script:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Reversing/01 BabyRev\Analysis and Scripts\Challenge 10 1
```

What to say -- explanation, step by step

1. I decompiled the target APK with jadx and found the flag check runs five independent segments, each comparing an MD5 hash of a candidate substring against its own fixed target hash.

2. Each segment also only allows a small, specific set of characters, which keeps a brute force of that segment alone fast even though the whole flag together would be far too slow to guess at once.

3. I wrote a Python script that brute-forces all five segments in parallel using multiprocessing, then joins the five recovered pieces into the final flag.

4. I typed the reassembled flag into the challenge site's flag box to solve it.

If an emulator isn't available

If asked how you got/confirmed the flag: "I decompiled the target with jadx, recovered the flag's five segments with a Python brute-force script (no exploit app was built or uploaded for this one), and typed the reassembled flag directly into the challenge site's own flag box." There is no APK submission or platform analysis log for this challenge -- don't claim one.

Result

Solved via constraint-based brute force; not separately documented as its own flag string elsewhere in this engagement's reports (see BabyRev's report or the master report for the fuller family write-up).